Advanced File System

Iskander Shaimerdinov | SE-2524
Software Design Patterns | Assignment 06 | 8 November 2026

Repository: github.com/IShaymerden/SDP_Assignment6

1. Purpose and object model

The application models a hierarchical file system entirely in memory. Three independent operations process the same tree: total byte size, configurable filename-and-size search, and structural statistics. The project uses Java 17 features, Maven and JUnit 5.

Element defines the common contract. **FileNode** stores an immutable name and non-negative long size. **DirectoryNode** owns named children. **AbstractNode** centralizes naming and parent ownership; its empty child list lets the walker treat a leaf uniformly. The sealed hierarchy restricts the model to supported node types.

2. Double dispatch and traversal

First, `element.accept(visitor)` selects the actual element implementation at runtime. Inside each concrete accept method, `visitor.visit(this)` selects the correct overload using the concrete type of this; virtual dispatch then chooses the actual visitor implementation. No runtime type checks or behavior-selecting casts are used.

TreeWalker is the only traversal owner. It uses an explicit stack for depth-first traversal. Elements only dispatch and visitors only process the current node. Each reachable node is visited once per operation, including empty directories. An iterative walk avoids Java call-stack growth on deeply nested trees.

Before each walk, reset() clears accumulated visitor state. Before each dispatch, beforeVisit(path, depth) supplies context. The selected root has depth 0 and path "."; descendant paths omit its name and use "/". Stored ancestors outside the chosen subtree never affect paths or depth. Search returns immutable, lexicographically sorted paths.

3. Demonstration structure

```
coursework/
  README.txt (600)     todo.txt (0)
  documents/
    notes.txt (1200)   report.pdf (4096)
    archive/draft.txt (300)
  src/
    Main.java (850)    Visitor.java (450)
  empty/
```

Seven files total 7,496 bytes. Five directories include the root and empty directory. The deepest node, documents/archive/draft.txt, is at depth 3.

Design decisions and Clean Code

4. Integrity and controlled state

DirectoryNode.addChild() rejects duplicate sibling names, an existing parent, and cycles before mutating anything. Cycle validation follows the destination directory's ancestors by identity. removeChild() releases ownership, allowing a detached subtree to move. getChildren() returns an immutable snapshot; names and sizes cannot be changed after construction.

Invalid sizes/names/attachments throw IllegalArgumentException; null required arguments throw NullPointerException. Names exclude blank values, ".", ".." and path separators. TotalSizeVisitor uses Math.addExact, so overflow raises ArithmeticException rather than silently wrapping.

Visitor instances are reusable through TreeWalker.walk(). Operations do not depend on execution order. After an exception, partial results must be discarded; the next walk resets state. The model is intentionally sequential: no tree mutation, re-entry or concurrent visitor sharing during traversal; filename predicates should be pure.

5. Five concrete Clean Code practices

Meaningful naming. SearchVisitor.filenameCriterion and minimumSize make the two search constraints explicit; getMaximumDepth() communicates the statistic directly.

Single Responsibility. DirectoryNode manages structural integrity, TreeWalker owns traversal, and each visitor contains one independent operation.

Small methods. DirectoryNode.addChild() delegates checks to validateAttachment() and performs the attachment only after validation succeeds.

Encapsulation. DirectoryNode.getChildren() returns List.copyOf(...); private fields and immutable names/sizes prevent callers from bypassing public invariants.

No type-checking chains. FileNode.accept() and DirectoryNode.accept() delegate to overloaded visit methods; polymorphism replaces instanceof, reflection and casts.

6. Extension boundary and trade-offs

The first commit implements the element contracts, lifecycle/context hooks, walker, size and search visitors. The second adds StatisticsVisitor and demo calls without changing FileNode, DirectoryNode, their accept methods, Element, Visitor or TreeWalker. Statistics count the root when it is a directory and include empty directories when determining maximum depth.

A new operation requires a visitor and client invocation. A new element type is less convenient: it requires updating the sealed hierarchy, adding a visit overload, implementing that overload in all visitors, and defining child/ownership semantics. This favors stable element types with frequently changing operations. Read-only accessors expose data needed by visitors while snapshots protect mutable collections.

Execution and verification

7. Actual application output

```
Visitor Pattern - Advanced File System
Root: coursework
Total size: 7496 bytes
Search: suffix=.txt, minimum size=600 bytes
  README.txt
  documents/notes.txt
Files: 7
Directories (including root): 5
Maximum depth (root = 0): 3
Repeated size calculation: 7496 bytes
```

Default search: case-sensitive suffix ".txt" AND size >= 600 bytes. The inclusive boundary includes README.txt at exactly 600 bytes. Changing arguments to `.java 500` returns only src/Main.java; size and statistics remain unchanged.

8. Automated verification

Test class	Count	Coverage
SearchVisitorTest	8	T3/T4/T6: filters, paths, sorting
StatisticsVisitorTest	5	T5/T6: counts and depth
TotalSizeVisitorTest	4	T2/T6: sum, empty, overflow
TraversalLifecycleTest	5	T7/R7: reuse, exactly once, deep tree
TreeIntegrityTest	9	T8: invariants and valid nesting
VisitorDispatchTest	2	T1: both overloads

BUILD SUCCESS - 33 tests, 0 failures, 0 errors, 0 skipped. The test fixture differs from the demo: four files sum to 1,920 bytes; four directories and maximum depth 3. A separate test visits 2,001 directory levels plus a file at depth 2,001.

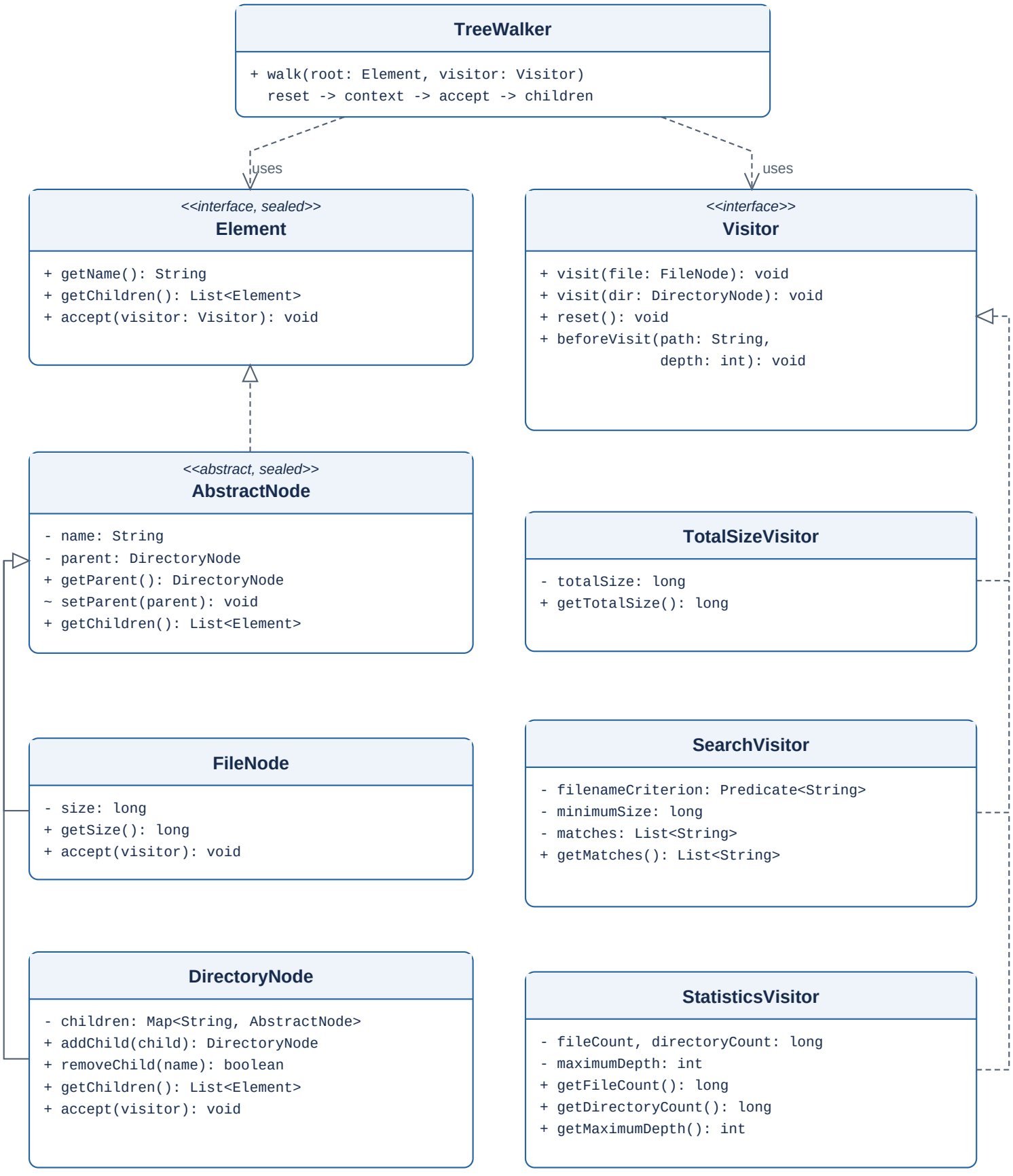
Run from the project root: `mvn test`. Application: `mvn compile`, then `java -cp target/classes edu.sdp.filesystem.Main`. Verified with Maven 3.9.12, JDK 25.0.1 and compiler --release 17 (class version 61), using an offline workspace copy of cached dependencies. A separate Java 17 runtime was not tested.

9. Sources and preparation

Sources: the supplied Assignment 06 specification and Almas Ospanov, Lecture 9, especially Visitor architecture, double dispatch, traversal and Clean Code. Implementation and documentation were prepared with OpenAI Codex assistance. The author should review and adapt the work and be able to trace the code and tests. The UML diagram follows as a separate appendix.

UML class diagram

Interfaces, ownership, traversal and three independent operations



Ownership: DirectoryNode owns 0..* AbstractNode children; each child has 0..1 parent.

Both concrete accept methods call visitor.visit(this). Visitors never traverse children.